Ders 18 Genişletilmiş Notu: Dinamik Programlama: 0-1 Sırt Çantası Probleminin Çözümü ve Robot ile Madeni Para Toplama

Ders 18 Genişletilmiş Notu: Dinamik Programlama: 0-1 Sırt Çantası Probleminin Çözümü ve Robot ile Madeni Para Toplama

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. 0-1 Sırt Çantası Bağıntısının Kurulması
- 2. Bağıntının Tablo Üzerinde Uygulanması
- 3. Seçilen Eşyaların Bulunması (Geri İzleme)
- 4. Robot ile Madeni Para Toplama Problemi
- 5. Robot Problemi İçin Bağıntı ve Sözde Kod
- 6. Robot Probleminde Rotanın Geri İzlenmesi
- Örnek Sorular

1. 0-1 Sırt Çantası Bağıntısının Kurulması

Bu kayıt, günün ilk oturumunda başlanan 0-1 sırt çantası (0-1 knapsack) probleminin devamıyla açılır. Önceki oturumda problem dinamik programlama (dynamic programming) yaklaşımıyla çözülmeye başlanmış, tablo kurulmuştu. Bu oturumda amaç, tabloyu dolduran kuralı tek bir bağıntı (recurrence) olarak yazmaktır. Hoca, sonunda tek bir bağıntı olacağını, ama önce her durum (case) için ayrı bağıntı kurmak gerektiğini söyledi.

Derste kullanılan örnekte sırt çantasının kapasitesi $W = 5$ kilogramdır ve dört eşya vardır:

Eşya i	Ağırlık w_i	Değer v_i
1	2	12
2	1	10
3	3	20
4	2	15

$P[i, w]$, ilk i eşya kullanılarak ve en fazla w kapasiteyle elde edilebilecek en büyük toplam değerdir. Tablonun satırları eşyalar (i), sütunları kapasitedir (w).

1.1 Durum 1: Eşya Alınır

İlk durum eşyanın alınmasıdır. Eşya alınıyorsa onun değeri v_i toplama eklenir. Hoca “alınan eşyaya neyi ekleyeceğim?” diye sordu. Bir öğrenci örneğe özel olarak “$i-1$, $w-2$” dedi. Hoca bunun o anki örnek için doğru olduğunu, ama genelleştirilirse $w - w_i$ yazılması gerektiğini

söyledi. Çünkü i. eşya alındıysa onun ağırlığı toplam kapasiteden çıkarılmalıdır. Kalan kapasitenin bir önceki satırdaki en iyi değerine gidilir. $i-1$ satırına gitmek, o ana kadar elde edilmiş en iyi seçeneklere bakmak demektir:

$$P[i-1,\ w-w_i]+v_i$$

1.2 Durum 2: Eşya Alınmaz

İkinci durum eşyanın alınmamasıdır. Hoca bunun çok daha kolay olduğunu söyledi. Bir öğrenci $P[i-1,w]$ cevabını verdi ve nedenini şöyle açıkladı: Eşya alınmadığı için kapasite azalmaz. Bu yüzden bir üst satıra çıkılır ve aynı kapasite sütununa bakılır. Orada o ana kadar elde edilen en büyük değer neyse o alınır.

1.3 Başlangıç Koşulları

Hoca iki durumu tek bağıntıda birleştirmeden önce konuşulması gereken başka durumlar olduğunu sordu. Bir öğrenci, matris hazırlandığına göre i ya da w sıfır olduğunda değerlerin baştan belli olduğunu söyledi. Hoca bunu başlangıç koşulu (initial condition) olarak yazdı:

$$P[0,w]=0, \qquad P[i,0]=0$$

Hiç eşya yoksa ya da kapasite sıfırsa elde edilebilecek değer sıfırdır.

1.4 Durum 3: Eşya Sığmaz

Bir başka öğrenci, eşyanın ağırlığının o anki kapasiteden fazla olduğu durumu söyledi. Bu durumda eşyayı alma ihtimali yoktur. Herhangi bir işlem yapmaya gerek kalmadan doğrudan bir üst satırdaki en büyük değer alınır: $P[i-1,w]$.

1.5 Birleşik Bağıntı

Böyle bir sorun yoksa iki durumdan büyük olanı seçilir. Hoca bağıntıyı şöyle yazdı:

$$P[i,w]=\begin{cases}0 & i=0 \text{ ya da } w=0\\ P[i-1,w] & w_i>w\\ \max\{P[i-1,\ w-w_i]+v_i,\ P[i-1,w]\} & \text{diğer}\end{cases}$$

Ders sonrası not: Altyazıda sığmama koşulu "V, I, W" diye geçiyor. Koşul ağırlıkla ilgilidir: $w_i > w$. Hoca da anlatımda "item'ın ağırlığı toplam ağırlıktan fazlaysa" diyor.

1.6 Bağıntının Tekrar Açıklanması

Bir öğrenci "max" kısmını karıştırdığını söyleyince hoca bağıntının üzerinden yeniden geçti. Eşya toplam kapasiteden ağırsa onunla ilgili bir işlem yapılmaz. Doğrudan bir önceki satırda kalınan en büyük değere gidilir. Eşyayı alma ihtimali varsa iki seçenek karşılaştırılır:

- Eşya alınırsa, eşyanın ağırlığı çıkarılır ve bir önceki satırda kalan ağırlık için en büyük değer neyse ona eşyanın değeri eklenir.
- Eşya alınmazsa, bir önceki satırın aynı sütunundaki değer alınır.
- İkisinden büyük olanla yola devam edilir.

2. Bağıntının Tablo Üzerinde Uygulanması

Hoca bağıntıyı bir önceki sayfadaki tablo üzerinde hücre hücre gösterdi.

2.1 $P[1,1]$ Neden Sıfır?

Hoca $P[1,1]$'in değerinin neden 0 olduğunu sordu. Bir öğrenci, şu an yalnız ilk eşyaya bakıldığını ve onun ağırlığının 1'den büyük olduğunu söyledi. Birinci eşyanın ağırlığı 2, kapasite 1'dir. $2 > 1$ olduğu için sığmama koşulu geçerlidir. Eşya alınmaz ve $P[0,1] = 0$ değeri aşağı taşınır.

2.2 $P[1,2]$ Hesabı

$P[1,2]$'de ağırlıkla ilgili bir sorun yoktur, $2 \leq 2$. Bu yüzden "max" yazılır. Hoca, öğrenciye formül üzerinden değil, durumlar üzerinden anlatmasını istedi:

- Eşya alınırsa: 12 değeri eklenir. 2 kiloluk kapasite kullanılmıştır, $2-2 = 0$ kalır. $P[0,0] = 0$ hücresine gidilir. Toplam $12 + 0 = 12$.
- Eşya alınmazsa: bir üstteki $P[0,2] = 0$ hücresine bakılır.
- $\max\{12, 0\} = 12$.

2.3 $P[2,3]$ Hesabı

Aynı öğrenciyle $P[2,3]$ hesaplandı. Önce ağırlık koşuluna bakılır: 2. eşyanın ağırlığı 1, kapasite 3, eşya sığar.

- Eşya alınırsa: $3 - 1 = 2$ kalır. Bir üst satırdaki $P[1,2] = 12$ hücresine gidilir. Eşyanın değeri 10 eklenir: $12 + 10 = 22$.
- Eşya alınmazsa: $P[1,3] = 12$.
- $\max\{22, 12\} = 22$.

Hoca temel prensibi şöyle özetledi: Ya üst satırın aynı sütunundaki değer büyükse o aşağı alınır, bu durumda eşya eklenmemiş olur. Ya da eşya eklenirse onun ağırlığı çıkarılır ve bir üst satırda kalan ağırlığın sütununa bakılır. Bu sayede her adımda, o ana kadarki eşyalar ve o kapasite kısıtı için en iyi sonuç hesaplanmış olur. Bir önceki adımda hesaplanan sonuç en iyi olduğu için ya o yukarıdan aşağı taşınır ya da eşyanın değeri, bir önceki satırdaki kalan ağırlığın en iyi değeriyle toplanır.

> **Hocanın vurgusu:** Her hücre, o ana kadarki eşyalar ve o kapasite için en iyi sonuçtur. Dinamik programlamanın temel prensibi budur.

2.4 Neden $w - w_i$? 32 Değeri Üzerinden Açıklama

Bir öğrenci $w - w_i$ kısmını neden yaptığımızı kavrayamadığını söyledi. Hoca, eşya alınıyorsa onun kilosunun toplam kapasiteden çıkarılması gerektiğini, çünkü bir önceki en iyiye bakarken o kiloya kadar olan en iyiye bakmak gerektiğini anlattı. Eşya alınmasaydı bu çıkarma yapılmazdı.

Öğrenci tam ikna olmayınca hoca tablodaki 32 değerini ($P[3,5]$) çizerek gösterdi. Öğrenci 32'nin 20 ile 12'nin toplamı olduğunu söyledi. Hoca açıklamayı şöyle sürdürdü:

- 3. eşyadayız ve 3. eşya alındı. Değeri 20, ağırlığı 3.
- $5 - 3 = 2$ kilo hak kalır.
- Bir önceki satırda 2 kiloya kadar olan en büyük değer $P[2,2] = 12$'dir.
- $20 + 12 = 32$.

2.5 Tamamlanmış Tablo

Önceki oturumda doldurulan tablonun tamamı aşağıdadır. Hoca son satırı geri izleme öncesinde "15, 25, 30, 37" diye yeniden yazdı. Tablo Python ile doğrulandı:

$i \backslash w$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22

$i \backslash w$	0	1	2	3	4	5
3	0	10	12	22	30	32
4	0	10	15	25	30	37

En büyük değer $P[4,5] = 37$'dir.

> **Ek bilgi (derste anlatılmadı):** Bu çözüm n eşya ve W kapasite için tabloda $(n+1)(W+1)$ hücre doldurur, yani $\Theta(nW)$ zaman ve yer kullanır. Süre girdinin uzunluğuna değil W'nin sayısal değerine bağlı olduğu için bu sınıra büyük olasılıkla sözde polinom (pseudo-polynomial) denir.

3. Seçilen Eşyaların Bulunması (Geri İzleme)

Hoca, şu ana kadar yalnız en büyük değerin bulunduğunu, bir şeyin eksik kaldığını sordu. Bir öğrenci, en büyük değere hangi elemanlar kullanılarak ulaşıldığının da bulunması gerektiğini söyledi. Hoca bunun doğru olduğunu ve bu bilgiye ihtiyaç olduğunu belirtti. Buna optimal çözümün yeniden kurulması (reconstruction of optimal solution) denir.

3.1 Hangi Hücreyle Karşılaştırılır?

Bir öğrenci yöntemi genel olarak doğru anlattı: Önce bulunulan hücrenin eşyasının kullanılıp kullanılmadığı öğrenilir. Kullanılmışsa eşyanın ağırlığı kapasiteden çıkarılır ve kalan kısım için devam edilir, sıfıra gelene kadar ilerlenir.

Hoca kullanılıp kullanılmadığının nasıl anlaşılacağını sordu. Öğrenci önce aynı satırdaki bir önceki hücreyle, yani 37 ile 30'u karşılaştırmayı önerdi. Hoca "tam tersi olabilir mi?" diye sordu ve doğru karşılaştırmanın bir üstteki hücreyle, yani 37 ile 32 arasında yapıldığını gösterdi.

Kural şöyledir:

- $P[i,w] = P[i-1,w]$ ise karşılaştırma doğru çıkmıştır. Bu, i. eşyanın kullanılmadığı anlamına gelir. Yalnız bir üst satıra çıkılır.
- $P[i,w] \neq P[i-1,w]$ ise i. eşya kullanılmıştır. Kapasiteden eşyanın ağırlığı çıkarılır ve $P[i-1,\ w-w_i]$ hücresine gidilir.

3.2 Örnekte Geri İzleme

- $P[4,5] = 37$, $P[3,5] = 32$. Eşit değil, 4. eşya alınmış. 4. eşyanın ağırlığı 2'dir. $5-2=3$ yapılır ve $P[3,3]$'e gidilir.
- $P[3,3] = 22$, $P[2,3] = 22$. Eşit, 3. eşya alınmamış. Yukarı çıkılır.
- $P[2,3] = 22$, $P[1,3] = 12$. Üstteki daha küçük, yani eşit değil. 2. eşya alınmış. 2. eşyanın ağırlığı 1'dir. $3-1=2$ yapılır ve $P[1,2]$'ye gidilir.
- $P[1,2] = 12$, $P[0,2] = 0$. Eşit değil, 1. eşya alınmış. Geriye eşya kalmaz.

Sonuç: 4, 2 ve 1 numaralı eşyalar alınır. Toplam ağırlık $2+1+2=5$, toplam değer $15+10+12=37$. Python ile yapılan geri izleme de aynı eşyaları verdi.

> **Ders sonrası not:** Bir öğrenci 4. eşyanın ağırlığını sorunca hoca önce "15" dedi, hemen ardından düzeltti: 15 eşyanın değeridir, ağırlığı 2 kilogramdır.

4. Robot ile Madeni Para Toplama Problemi

Hoca bu saatin alıştırması (exercise) olarak yeni bir probleme geçti: robot ile madeni para toplama problemi (robot coin collection problem).

4.1 Problemin Tanımı

Izgaralara (grid) bölünmüş bir alan vardır. Bunu bir harita gibi düşünmek gerekir. Bazı hücrelerde madeni para (coin) vardır. Robot sol üst köşeden girer, sağ alt köşeden çıkar. Her

hareketinde ya sağa ya da aşağı gider; ikisinden birini seçmek zorundadır. Harita robota verilmiştir. Robot bu haritada en fazla parayı toplamak ister ve yola çıkmadan önce izleyeceği rotayı belirlemek ister.

Derste kullanılan ızgara 5 satır ve 6 sütundur. Paralar şu hücrelerdedir (satır, sütun): (1,5), (2,2), (2,4), (3,4), (3,6), (4,3), (4,6), (5,1), (5,5).

Izgara aşağıdaki gibidir ($c_{ij} = 1$: hücrede para var):

$i\backslash j$	1	2	3	4	5	6
1	0	0	0	0	1	0
2	0	1	0	1	0	0
3	0	0	0	1	0	1
4	0	0	1	0	0	1
5	1	0	0	0	1	0

4.2 Robotun Gözüyle Düşünmek

Hoca önce formül istemediğini, robotun gözüyle bakınca nasıl ilerleneceğini sordu. Bir öğrenci, robot yukarı gidemediği için bir satırı bitirmeden aşağı geçmenin geri dönüşü olmadığını söyledi, ama toplama işlemini nasıl yapacağını bulamadı. Hoca her parayı $+1$ olarak düşünmeyi önerdi ve yanına 5'e 6'lık ikinci bir tablo çizerek yardımcı oldu.

4.3 Tablonun Doldurulması

- **İlk satır:** Robot ilk satırda yalnız soldan gelebilir. Değerler 0, 0, 0, 0 diye ilerler. Para olan (1,5) hücresinde değer 1 olur. Sağa gitseydi değer 1 olarak devam ederdi.
- **İlk sütun:** Robot ilk sütunda yalnız yukarıdan gelebilir. Değerler 0, 0, 0, 0 ilerler. (5,1)'de para olduğu için değer 1 olur.
- **Diğer hücreler:** Öğrenci ikinci satırı doldururken bir yerde "yana yine 2 diye gidecek, ya da 3 olacak" diye tereddüt etti. Hoca sırt çantasındaki mantığı hatırlattı: Robot o hücreye yukarıdan da gelmiş olabilir, soldan da. Öğrenci önce "minimumunu alacağız" dedi. Hoca en fazla parayı topladığımız için maksimumun peşinde olduğumuzu hatırlattı. Böylece ikinci satır 0, 1, 1, 2, 2, 2 oldu.
- Bir başka öğrenci üçüncü satırı doldurdu ve kuralı kendisi şöyle söyledi: "Üstteki ve soldakinden büyük olan artı kendisi." Örneğin (3,4)'te üstteki 2 alınır ve hücredeki para eklenir: 3.
- Aynı öğrenci dördüncü satırı da söyledi. Son satırı hoca yazdı. (5,2)'de para olmadığı için oraya 2 yazılmaz, 1 kalır.

Doldurulan tablo Python ile doğrulandı:

$i\backslash j$	1	2	3	4	5	6
1	0	0	0	0	1	1
2	0	1	1	2	2	2
3	0	1	1	3	3	4
4	0	1	2	3	3	5
5	1	1	2	3	4	5

Robot en fazla $F[5, 6] = 5$ para toplayabilir.

4.4 Zaman ve Yer Karmaşıklığı

Hoca bu problemi dinamik programlamayla çözmenin maliyetini sordu. Bir öğrenci, zaman karmaşıklığının (time complexity) matrisin genişliği çarpı yüksekliği olduğunu, örnekte $6 \times 5 = 30$ ettiğini söyledi. Her hücre bir kez hesaplanır. Yer karmaşıklığı (space complexity) için öğrenci önce emin olamadı. Hoca kullanılan alanın belli olduğunu hatırlattı ve öğrenci bunun da aynı, $n \times m$ olduğunu söyledi:

- Zaman: $\Theta(nm)$.
- Yer: $\Theta(nm)$.

Hoca, bazı durumlarda birden fazla yol çıktığını ve geriye dönüşte ayrıca bir hesaplama yapıldığını, bu kısmın yeniden kurmaya (reconstruction) ait olduğunu söyledi. Konuşulan karmaşıklıklar en büyük değerin hesaplanmasına aittir.

> **Ek bilgi (derste anlatılmadı):** Geri izleme her adımda bir satır yukarı ya da bir sütun sola gider. Bu yüzden tek bir rotayı geri izlemek $n+m-2$ adım, yani $\Theta(n+m)$ sürer; bu, tablo doldurmanın $\Theta(nm)$ maliyetinin yanında küçüktür.

5. Robot Problemi İçin Bağıntı ve Sözde Kod

Hoca, para matrisine C, doldurulan tabloya F demeyi önerdi. Başlangıç noktası sol üst köşe olduğu için önce $F[1,1]$'e $C[1,1]$ atanır.

Bağıntı şöyledir:

$$F[1,1] = c_{11}$$

$$F[1,j] = F[1,j-1] + c_{1j}, \qquad F[i,1] = F[i-1,1] + c_{i1}$$

$$F[i,j] = \max\{F[i-1,j],\ F[i,j-1]\} + c_{ij}$$

Sonuç $F[n,m]$'dir.

Hoca sözde kodu (pseudocode) öğrencilerle birlikte yazdı. Önce ilk satır tek bir döngüyle tamamlanır. Sonra iç içe döngü kurulur: Dış döngü her satırın ilk hücresini yukarıdan doldurur, iç döngü o satırın sütunlarında ilerler. Kritik nokta iç döngüdeki satırdır. Bir öğrenci orada üst hücre ile sol hücrenin karşılaştırılacağını ve büyük olanın alınacağını söyledi. Hoca buna $C[i,j]$'nin ekleneceğini tamamladı. Döngülerden çıkınca $F[n,m]$ döndürülür.

```
F[1,1] <- C[1,1]
for j <- 2 to m do
    F[1,j] <- F[1,j-1] + C[1,j]
for i <- 2 to n do
    F[i,1] <- F[i-1,1] + C[i,1]
    for j <- 2 to m do
        F[i,j] <- max(F[i-1,j], F[i,j-1]) + C[i,j]
return F[n,m]
```

Hoca, tablonun normal bir matrisi gezer gibi satır satır, aşağıya doğru adım adım doldurulduğunu belirtti. Yine önceki değerlerden yararlanılır: Üstteki ya da soldakinden büyük olanı aşağı taşınır.

6. Robot Probleminde Rotanın Geri İzlenmesi

6.1 Geri İzleme Kuralı

Rota bulunurken son noktadan başlanır. Üstteki ve soldaki hücreye bakılır, hangisi büyükse o tarafa ilerlenir ve robotun o taraftan geldiği işaretlenir.

Bir öğrenci bunu kendisi fark etti: Tersten bakarsak en büyük değerin olduğu son hücredeyiz. Geri giderken her zaman ya soldaki ya da yukarıdaki hücreye gitmek zorundayız ve ikisinden büyük olanına gitmeliyiz. Hoca bu tespitin doğru olduğunu söyledi. Buradaki durum sırt çantasından biraz farklıdır. Sırt çantasında yalnız üstteki hücreyle eşitlik kontrol edilir. Robotta ise eşitlik olmayan durumlar da vardır ve nereye gidileceğini hangisinin büyük olduğu belirler.

6.2 Örnekte Geri İzleme

Hoca tablo üzerinde geri izlemeyi gösterdi. $F[5,6] = 5$'ten başlanır. Soldaki $F[5,5] = 4$, üstteki $F[4,6] = 5$'tir. Büyük olan üstteki olduğu için yukarı çıkılır. $F[4,6] = 5$'te soldaki $F[4,5] = 3$, üstteki $F[3,6] = 4$'tür. Hoca "4'e doğru ilerlememiz gerekiyor, çünkü bir tane coin eklemişiz" dedi. Böyle devam edilir.

Sonunda (2,2) hücresine gelinir. Burada değer 1'dir ve hem üstteki $F[1,2]$ hem soldaki $F[2,1]$ 0'dır. İki komşu eşit olduğu için robot buraya iki yoldan da gelmiş olabilir. Bu yüzden iki farklı en iyi rota vardır.

Python ile bütün en iyi rotalar listelendi (R: sağa, D: aşağı). İki rota vardır, ikisi de 5 para toplar:

- $R\,D\,R\,R\,D\,R\,R\,D\,D$: (1,1) → (1,2) → (2,2) → (2,3) → (2,4) → (3,4) → (3,5) → (3,6) → (4,6) → (5,6)
- $D\,R\,R\,R\,D\,R\,R\,D\,D$: (1,1) → (2,1) → (2,2) → (2,3) → (2,4) → (3,4) → (3,5) → (3,6) → (4,6) → (5,6)

Toplanan paralar her iki rotada da (2,2), (2,4), (3,4), (3,6), (4,6) hücrelerindedir.

> **Ders sonrası not:** Hoca iki rotayı sözlü olarak "önce sağa gidip sonra aşağı inip …" ve "önce aşağı sonra sağa …" diye özetledi, ama altyazıdaki adım dizileri tam olarak tutarlı değil. Tablo ve Python sonucu yukarıdaki iki rotayı verir; iki rota yalnız ilk iki adımda ayrışır.
>
> **Hocanın vurgusu:** En iyi rota birden fazla olabilir. Geri izlerken iki komşu eşit çıkarsa iki ayrı yol ortaya çıkar.

6.3 Sırt Çantasıyla Fark: Hücre Atlanabilir mi?

Hoca bu problemde herhangi bir noktayı almamak, atlamak gibi bir olasılık olup olmadığını sordu ve kendisi cevapladı: Teknik olarak atlayarak gidilmez. Robot her adımda mutlaka ya yukarıdan ya soldan bir ızgara hücresine uğrar. Sırt çantasında ise atlamalı gidiliyordu: Her zaman bir önceki satıra uğranıyordu, ama bir önceki sütun olmak zorunda değildi. Yani sırt çantasında her noktayı gezmek gerekmez, robotta ise komşu hücrelere mutlaka uğranır.

> **Ek bilgi (derste anlatılmadı):** Robot yalnız sağa ve aşağı gidebildiği için bir hücreye yalnız üstünden ya da solundan gelinebilir. Çapraz hareket de serbest olsaydı, bağıntıdaki maksimuma büyük olasılıkla $F[i-1, j-1]$ de eklenirdi.

Örnek Sorular

Bu derste hazır kare ve video yoktur. Aşağıdaki sorular hocanın derste sözlü olarak sorduğu sorulardır.

Soru 1 (derste sözlü, 00:08:01): Sırt çantası tablosunda $P[1,1]$'in değeri neden 0'dır? Hangi koşuldan dolayı?

> **Çözüm (derste verildi):** 1. eşyanın ağırlığı 2, kapasite 1'dir. $w_1 = 2 > 1$ olduğu için sığmama koşulu geçerlidir. $P[1,1] = P[0,1] = 0$.

Soru 2 (derste sözlü, 00:09:00 ve 00:11:17): $P[1,2]$ ve $P[2,3]$ hücrelerini durumlar (eşya alınır / alınmaz) üzerinden hesaplayın.

> **Çözüm (derste verildi):** $P[1,2] = \max\{12 + P[0,0],\ P[0,2]\} = \max\{12, 0\} = 12$. $P[2,3] = \max\{10 + P[1,2],\ P[1,3]\} = \max\{22, 12\} = 22$.

Soru 3 (derste sözlü, 00:14:58): $P[3,5] = 32$ değerine hangi toplamdan ulaşıldı?

> **Çözüm (derste verildi):** 3. eşya alınmıştır (değer 20, ağırlık 3). $5 - 3 = 2$ kalır. $P[2,2] = 12$. $20 + 12 = 32$.

Soru 4 (derste sözlü, 00:16:29): En büyük değeri bulduk. Bu problemde bulmamız gereken başka ne var?

Çözüm (derste verildi): Hangi eşyaların seçildiği. Geri izlemeyle 4, 2 ve 1 numaralı eşyalar bulunur (toplam değer 37).

Soru 5 (derste sözlü, 00:18:31): Geri izlemede ilk karşılaştırmayı hangi hücreyle yapmalıyız?

Çözüm (derste verildi): Aynı satırdaki soldaki hücreyle (30) değil, bir üstteki hücreyle (32) karşılaştırılır. $P[i, w] \neq P[i-1, w]$ ise i. eşya alınmıştır.

Soru 6 (derste sözlü, 00:26:30): Robot, yalnız sağa ve aşağı giderek en fazla parayı toplamak ve rotayı yola çıkmadan belirlemek istiyor. Nasıl bir mekanizma kurmalıyız?

Çözüm (derste verildi): Her hücreye yukarıdan ya da soldan gelinir. $F[i, j] = \max\{F[i-1, j], F[i, j-1]\} + c_{ij}$ ile tablo doldurulur. Örnekte en fazla 5 para toplanır.

Soru 7 (derste sözlü, 00:33:47): Bu problemi dinamik programlamayla çözmenin zaman ve yer karmaşıklığı nedir?

Çözüm (derste verildi): Zaman $\Theta(nm)$, örnekte $5 \times 6 = 30$ hücre. Yer de $\Theta(nm)$.

Soru 8 (derste sözlü, 00:42:54): Robot probleminde herhangi bir hücreyi atlamak, almamak gibi bir olasılık var mı?

Çözüm (derste verildi): Hayır. Robot her adımda ya yukarıdan ya soldan komşu bir hücreye mutlaka uğrar. Sırt çantasında ise satırlar arasında sütun atlanabiliyordu.

Dersin Toparlaması

Bu derste 0-1 sırt çantası probleminin dinamik programlama bağıntısı üç durumla kuruldu: eşya sığmazsa üst satırın değeri taşınır, sığarsa "alınır" ve "alınmaz" seçeneklerinden büyüğü seçilir. Başlangıçta sıfırıncı satır ve sütun 0'dır. Tablo hücre hücre doldurularak en büyük değer 37 bulundu. Sonra son hücreden başlayıp her hücre bir üstündekiyle karşılaştırılarak seçilen eşyalar (4, 2 ve 1) geri izlendi. Ardından robot ile madeni para toplama problemi aynı mantıkla çözüldü: her hücreye yukarıdan ya da soldan gelinir, büyük olan alınıp hücredeki para eklenir. Sözde kod yazıldı, zaman ve yer karmaşıklığının $\Theta(nm)$ olduğu görüldü ve geri izlemede iki farklı en iyi rota çıktığı gösterildi.

Hoca dinamik programlama mantığının oturmasının zaman alabileceğini, bu yüzden uygulama saatlerinde bol örnek çözüldüğünü söyledi. Gelecek hafta düzenleme uzaklığı (edit distance), literatürde Levenshtein uzaklığı olarak da geçen algoritma, ardından en uzun ortak alt dizi (longest common subsequence) problemi işlenecek. Bunlar bu haftaki problemlere benzer işlerdir. Daha sonra graflara ve graf algoritmalarının analizine geçilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.